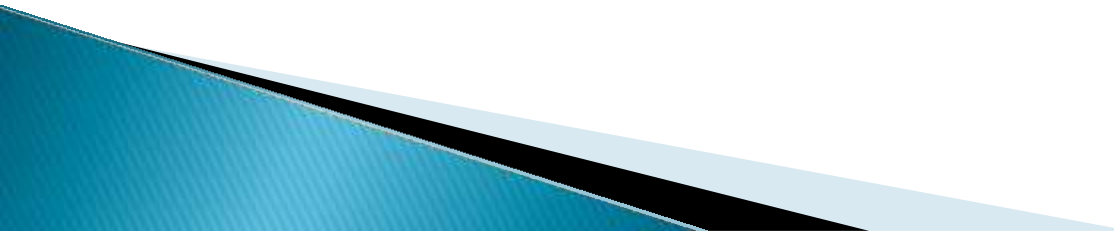


# **UNIT 5**

## **SORTING, SEARCHING AND HASHING**

# What is Sorting – Definition

- ▮ **Sorting** is the process of arranging data into meaningful order so that **you** can analyze it more effectively.
  - ▮ Often in real life, we are supposed to arrange data in a particular order. For instance, during our school days, we are told to stand in the queue based on our heights.
  - ▮ Another example is the attendance register at school/college which contains our names arranged in alphabetical order.
- 

These data arrangements give easier access to data for future use for ex. finding "Joe" in an attendance register of 100 students.

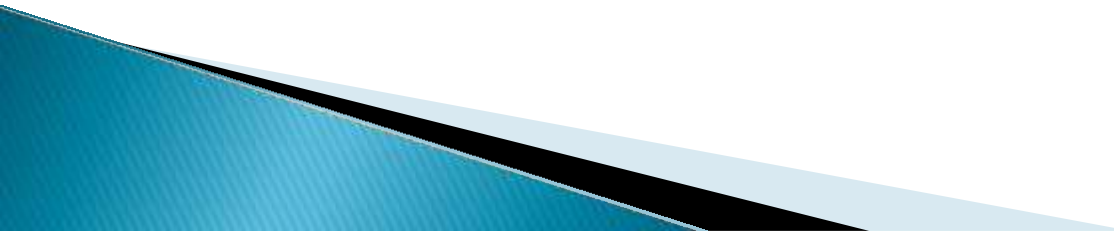
**The arrangement of data in a particular order is called as sorting of the data by that order.**

- Two of the most commonly used orders are:
  - Ascending order
  - Descending order

▮ **Ascending order** : the data in ascending order, we try to arrange the data in a way such that each element is in some way “smaller than” its successor.

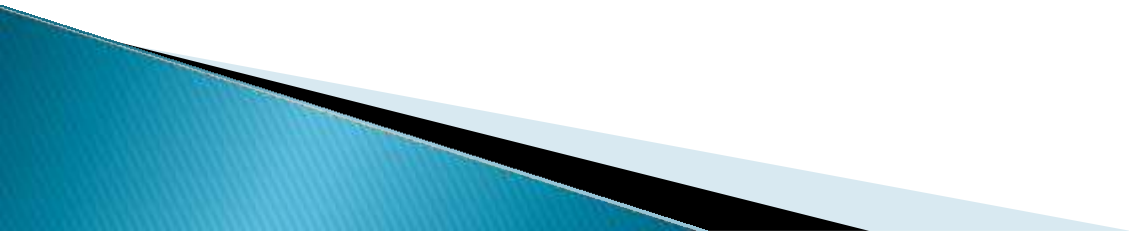
▮ As a simple example, the numbers 1, 2, 3, 4, 5 are sorted in ascending order. Here, the “smaller than” relation is actually the “<” operator. As can be seen,  $1 < 2 < 3 < 4 < 5$ .

▮ **Descending order**: descending order is the exact opposite of ascending order. Given a data that is sorted in ascending order, reverse it and you will get the data in descending order.



# Types of Sorting

1. Bubble Sorting
2. Insertion Sorting
3. Quick Sorting
4. Merge Sorting
5. Selection Sorting
6. Shell Sorting

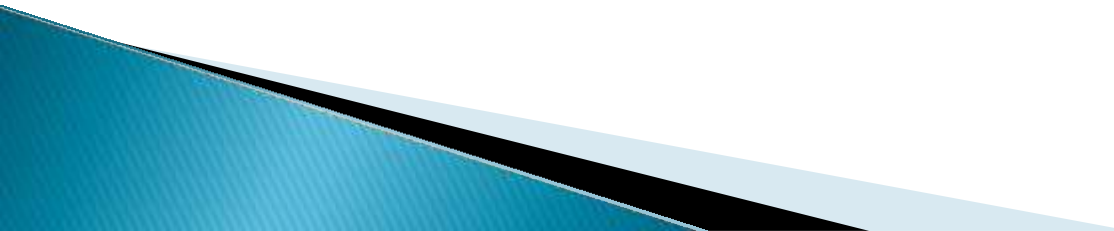


# 1. Bubble Sort

**Bubble Sort** is a **sorting algorithm** where we repeatedly iterate through the array and swap adjacent elements that are unordered. We repeat this until the array is sorted.



Bubble sort is one of the easiest sorting techniques in programming and it is very simple to implement. It just simply compares the current element with the next element and swaps it, if it is greater or less, depending on the condition. It gives quite accurate results. Each time an element is compared with all other elements till it's final place is found is called a pass. **Bubble sort gets its name because it filters out the elements at the top of the array like bubbles on water.**



# How Bubble Sort Works?

■ We take an unsorted array for our example.



Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this –



Next we compare 33 and 35. We find that both are in already sorted positions.



Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.



We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –



To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –



Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sort learns that an array is completely sorted.



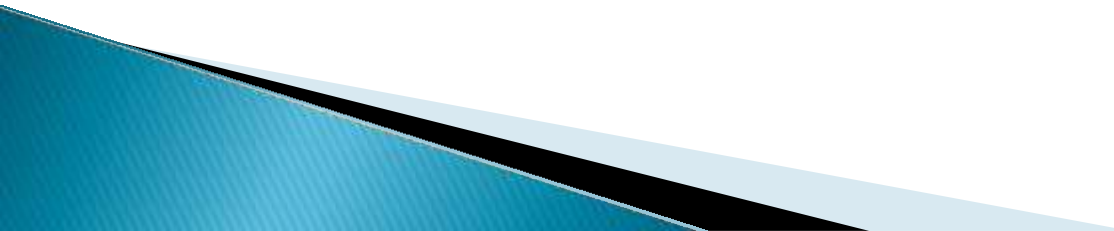
Now we should look into some practical aspects of bubble sort.

# Bubble sort algorithm

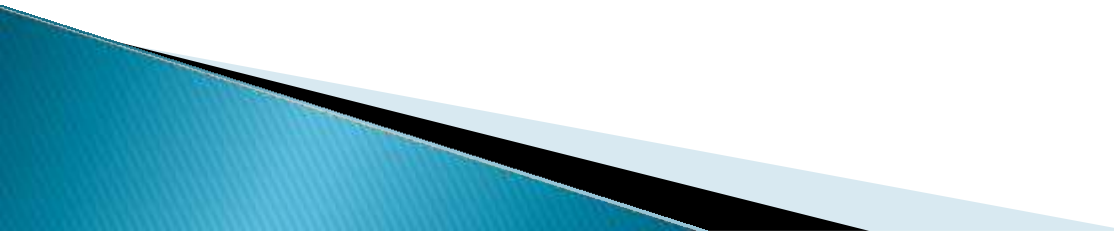
- 1) Start at index zero, compare the element with the next one ( $a[0]$  &  $a[1]$  ( $a$  is the name of the array)), and swap if  $a[0] > a[1]$ . Now compare  $a[1]$  &  $a[2]$  and swap if  $a[1] > a[2]$ . Repeat this process until the end of the array. After doing this, the largest element is present at the end. **This whole thing is known as a pass.** In the first pass, we process array elements from  $[0, n-1]$ .
- 2) Repeat step one but process array elements  $[0, n-2]$  because the last one, i.e.,  $a[n-1]$ , is present at its correct position. After this step, the largest two elements are present at the end.
- 3) Repeat this process  $n-1$  times

## Example

```
#include <stdio.h>
void bubble_sort(int a[], int n)
{
    int i = 0, j = 0, tmp;
    for (i = 0; i < n; i++) { // loop n times - 1 per element
        for (j = 0; j < n - i - 1; j++) { // last i elements are
            sorted already
            if (a[j] > a[j + 1]) { // swop if order is broken
                tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
        }
    }
}
```



```
void main()
{
int a[100], n, i, d, swap;
printf("Enter number of elements in the array:\n");
scanf("%d", &n);
printf("Enter %d integers\n", n);
for (i = 0; i < n; i++)
scanf("%d", &a[i]);
bubble_sort(a, n);
printf("Printing the sorted array:\n");
for (i = 0; i < n; i++)
printf("%d\n", a[i]);
}
```



# Output

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program

Enter number of elements in the array:

5

Enter 5 integers

9

8

3

2

11

Printing the sorted array:

2

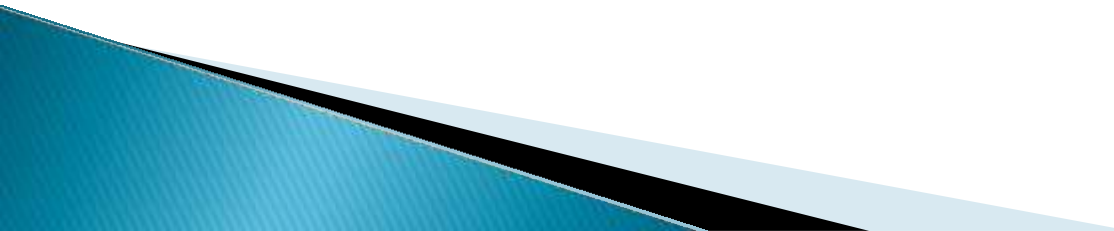
3

8

9

11

# Selection Sort

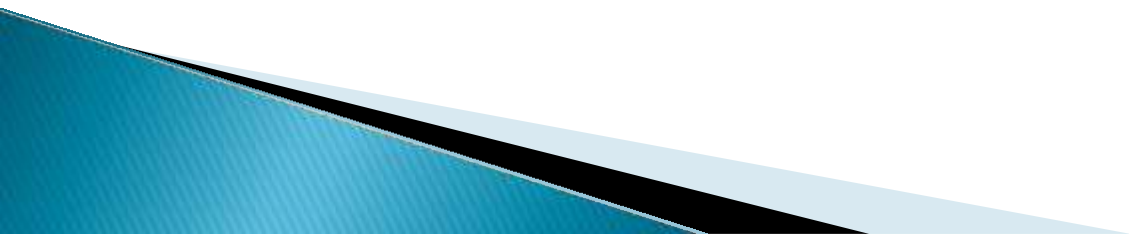
- ▮ In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array.
  - ▮ First, find the smallest element of the array and place it on the first position.
  - ▮ Then, find the second smallest element of the array and place it on the second position. The process continues until we get the sorted array.
- 

- ▮ The array with  $n$  elements is sorted by using  $n-1$  pass of selection sort algorithm.
- ▮ Selection sort is a simple sorting algorithm.
- ▮ This sorting algorithm is an in-place comparison-based algorithm in which the list is divided into two parts, the sorted part at the left end and the unsorted part at the right end.
- ▮ Initially, the sorted part is empty and the unsorted part is the entire list.

▮ The algorithm maintains two subarrays in a given array.

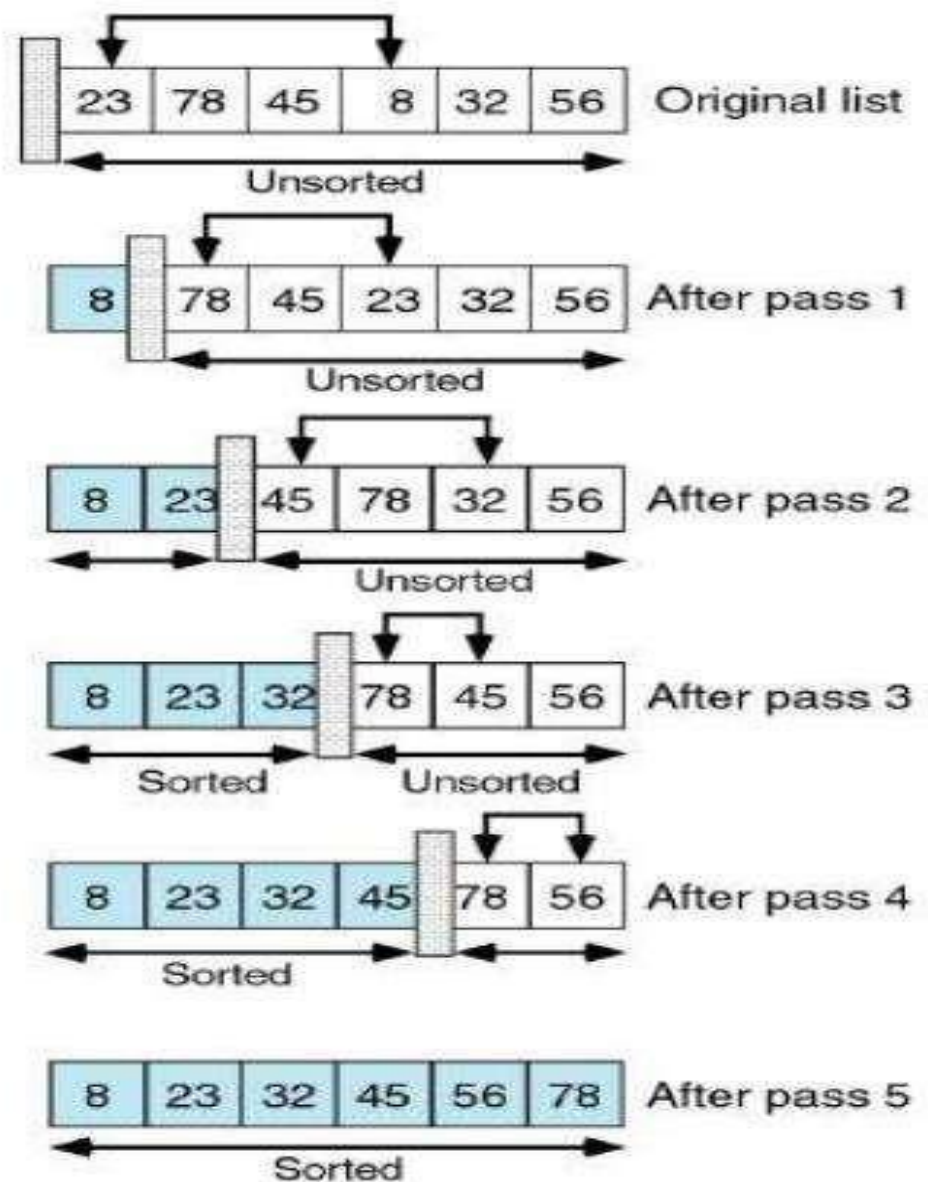
- The subarray which is already sorted.
- Remaining subarray which is unsorted

▮ In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.



# Selection sort

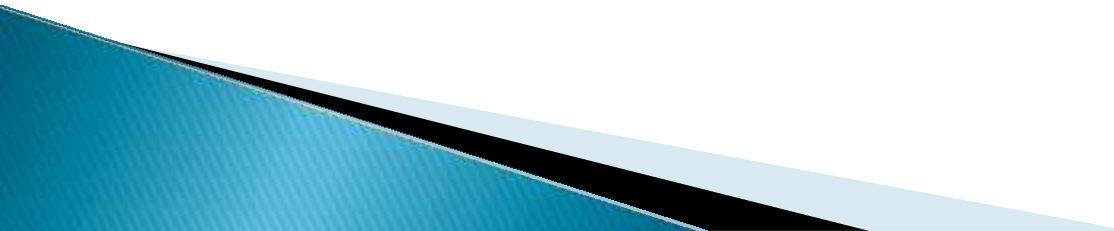
Example:  
small-to-large sort



## Step by Step Process

The selection sort algorithm is performed using the following

steps...

- **Step 1** – Select the first element of the list (i.e., Element at first position in the list).
  - **Step 2:** Compare the selected element with all the other elements in the list.
  - **Step 3:** In every comparision, if any element is found smaller than the selected element (for Ascending order), then both are swapped.
  - **Step 4:** Repeat the same procedure with element in the next position in the list till the entire list is sorted.
- 

# Example :

```
#include <stdio.h>
#include <conio.h>
// function to swap the the position of two elements
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
void selectionSort(int a[], int n)
{
    int i,j;
    for (i = 0; i < n - 1; i++)
    {
        int min_idx = i;
        for (j = i + 1; j < n; j++)
        {
            if (a[j] < a[min_idx])
                min_idx = j;
        }
        swap(&a[min_idx], &a[i]);
    }
}
```

```

void printArray(int array[], int n)
{
    int i;
    for (i = 0; i < n; ++i)
    {
        printf("%d ", array[i]);
    }
    printf("\n");
}

void main()
{
    int a[100], n, i;
    printf("\nEnter Number of array size:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("\nEnter array element:");
        scanf("%d", &a[i]);
    }
    selectionSort(a, n);
    printf("\nSorted array in Ascending Order:\n");
    printArray(a, n);
}

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameski

```

Enter Number of array size:5

Enter array element:80

Enter array element:8

Enter array element:1

Enter array element:0

Enter array element:9

Sorted array in Ascending Order:
0 1 8 9 80

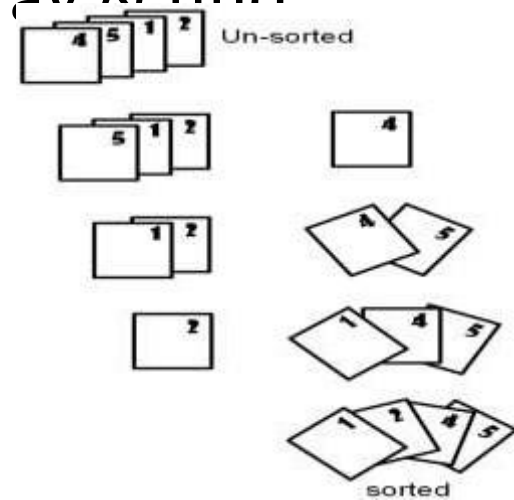
```

# Insertion Sorting

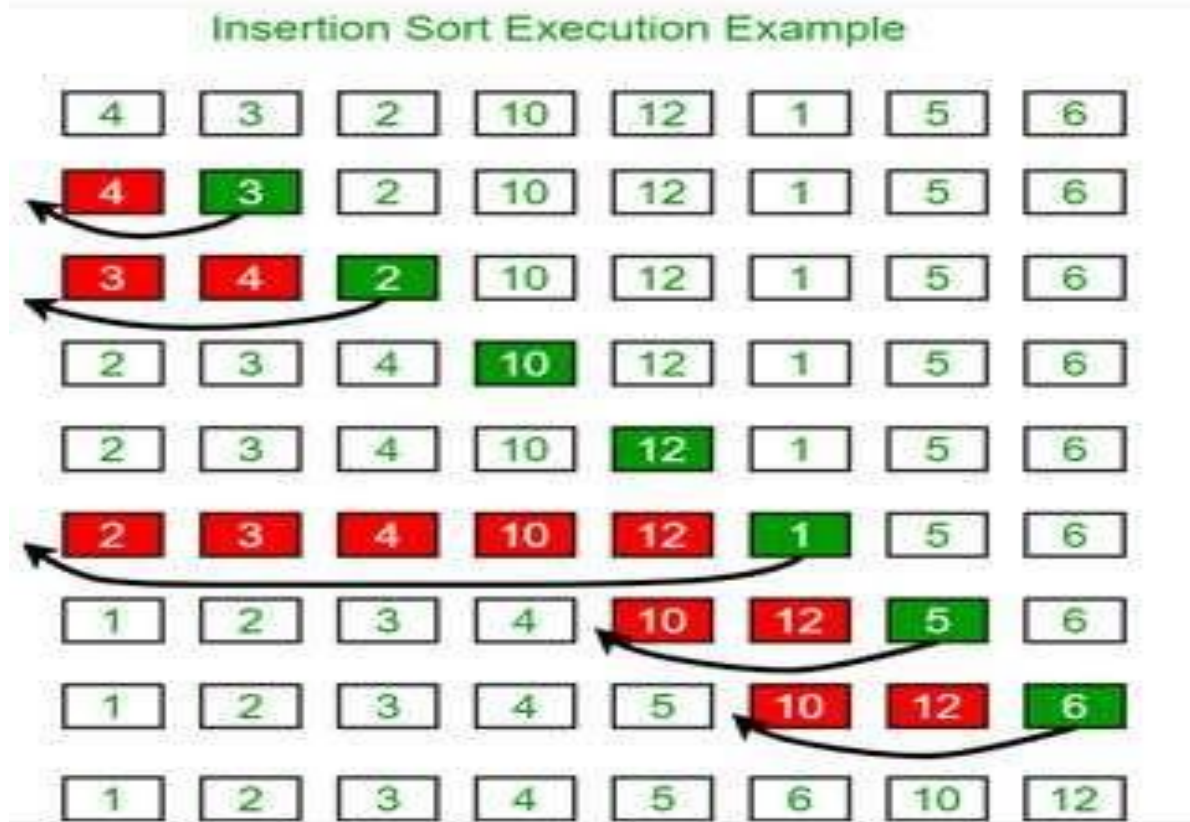
Insertion Sort in C is a simple and efficient sorting algorithm, that creates the final sorted array one element at a time. It is usually implemented when the user has a small data set.

Insertion Sort is a sorting algorithm where the array is sorted by taking one element at a time.

The principle behind insertion sort is to take one element, iterate through the sorted array & find its correct position in the sorted array.



Insertion Sort works in a similar manner as we arrange a deck of cards.



### **Another Example:**

**12, 11, 13, 5, 6**

Let us loop for  $i = 1$  (second element of the array) to 4 (last element of the array)

$i = 1$ . Since 11 is smaller than 12, move 12 and insert 11 before 12

**11, 12, 13, 5, 6**

$i = 2$ . 13 will remain at its position as all elements in  $A[0..i-1]$  are smaller than 13

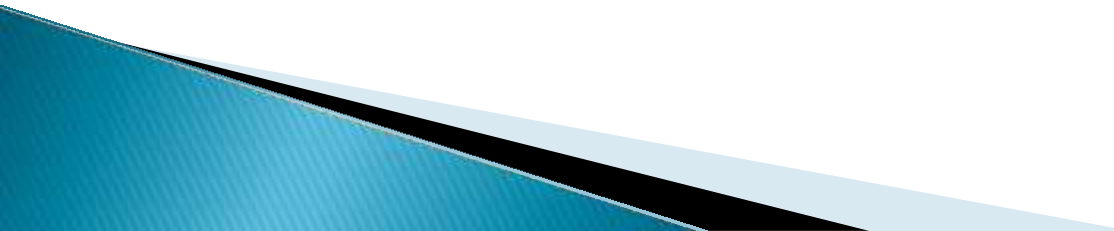
**11, 12, 13, 5, 6**

$i = 3$ . 5 will move to the beginning and all other elements from 11 to 13 will move one position ahead of their current position.

**5, 11, 12, 13, 6**

$i = 4$ . 6 will move to position after 5, and elements from 11 to 13 will move one position ahead of their current position.

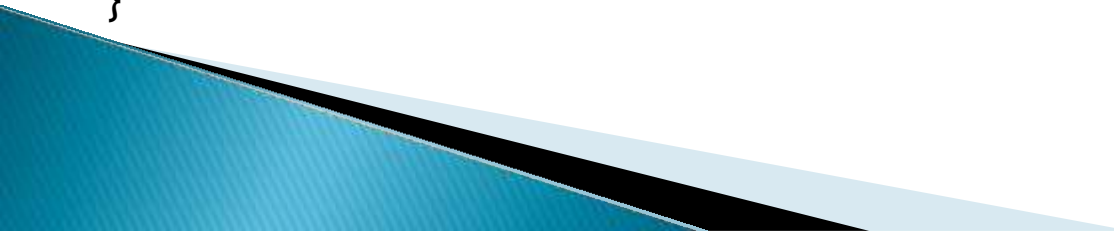
**5, 6, 11, 12, 13**



# Example

```
#include<stdio.h>
#include<conio.h>
#define MAX_SIZE 5
void insertion(int[]);
void main()
{
    int arr_sort[MAX_SIZE], i; printf("Simple Insertion Sort Example \n");
    printf("\nEnter %d Elements for Sorting\n", MAX_SIZE);
    for (i = 0; i < MAX_SIZE; i++)
    {
        scanf("%d", &arr_sort[i]);
    }
    printf("\nYour Data :");
    for (i = 0; i < MAX_SIZE; i++)
    {
        printf("\t%d", arr_sort[i]);
    }
    insertion(arr_sort);
    getch();

}
```



```

void insertion(int arr[])
{
    int i, j, a, t;
    for (i = 1; i < MAX_SIZE; i++)
    {
        t = arr[i]; j = i - 1;
        while (j >= 0 && arr[j] > t)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = t;
        printf("\nIteration %d : ", i);
        for (a = 0; a < MAX_SIZE; a++)
        {
            printf("\t%d", arr[a]);
        }
    }
    printf("\n\nSorted Data :");
    for (i = 0; i < MAX_SIZE; i++)
    {
        printf("\t%d", arr[i]);
    }
}

```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program: TC

### Simple Insertion Sort Example

Enter 5 Elements for Sorting

908  
1  
7  
700  
40

|               |     |     |     |     |     |
|---------------|-----|-----|-----|-----|-----|
| Your Data :   | 908 | 1   | 7   | 700 | 40  |
| Iteration 1 : | 1   | 908 | 7   | 700 | 40  |
| Iteration 2 : | 1   | 7   | 908 | 700 | 40  |
| Iteration 3 : | 1   | 7   | 700 | 908 | 40  |
| Iteration 4 : | 1   | 7   | 40  | 700 | 908 |
| Sorted Data : | 1   | 7   | 40  | 700 | 908 |

# Merge Sort

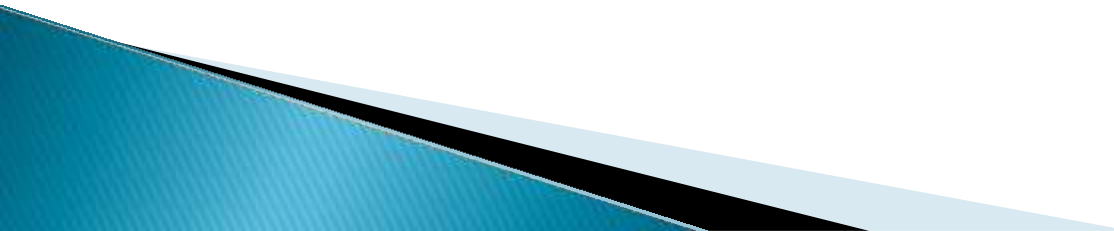
▮ The operation of sorting is closely related to process of merging

▮ Merge Sort is a divide and conquer algorithm

▮ It is based on the idea of breaking down a list into several sub-lists until each sub list consists of a single element Merging those sub lists in a manner that results into a sorted list Procedure

- Divide the unsorted list into  $N$  sub lists, each containing 1 element
- Take adjacent pairs of two singleton lists and merge them to form a list of 2 elements.  $N$  will now
- convert into  $N/2$  lists of size 2
- Repeat the process till a single sorted list of obtained

▮ Time complexity is  $O(n \log n)$



# Working of Merge sort Algorithm

Now, let's see the working of merge sort Algorithm.

To understand the working of the merge sort algorithm, let's take an unsorted array. It will be easier to understand the merge sort via an example.

Let the elements of array are -

|    |    |    |   |    |    |    |    |
|----|----|----|---|----|----|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|----|----|----|---|----|----|----|----|

According to the merge sort, first divide the given array into two equal halves. Merge sort keeps dividing the list into equal parts until it cannot be further divided.

As there are eight elements in the given array, so it is divided into two arrays of size 4.

|        |    |    |    |   |    |    |    |    |
|--------|----|----|----|---|----|----|----|----|
| divide | 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|--------|----|----|----|---|----|----|----|----|

Now, again divide these two arrays into halves. As they are of size 4, so divide them into new arrays of size 2.

|        |    |    |    |   |    |    |    |    |
|--------|----|----|----|---|----|----|----|----|
| divide | 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|--------|----|----|----|---|----|----|----|----|

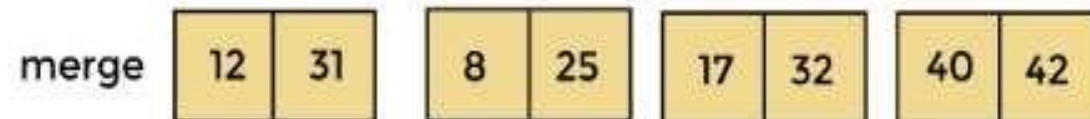
Now, again divide these arrays to get the atomic value that cannot be further divided.



Now, combine them in the same manner they were broken:

In combining, first compare the element of each array and then combine them into another array in sorted order.

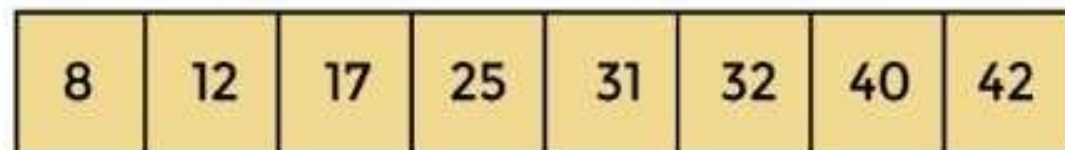
So, first compare 12 and 31, both are in sorted positions. Then compare 25 and 8, and in the list of two values, put 8 first followed by 25. Then compare 32 and 17, sort them and put 17 first followed by 32. After that, compare 40 and 42, and place them sequentially.



In the next iteration of combining, now compare the arrays with two data values and merge them into an array of found values in sorted order.



Now, there is a final merging of the arrays. After the final merging of above arrays, the array will look like -



# Merge Sort Algorithm

MergeSort(arr[], l, r), where l is the index of the first element & r is the index of the last element.

▮ If  $l < r$

▮ 1. Find the middle index of the array to divide it in two halves:  $m = (l+r)/2$

▮ 2. Call MergeSort for first half: mergeSort(array, l, m)

▮ 3. Call mergeSort for second half: mergeSort(array, m+1, r)

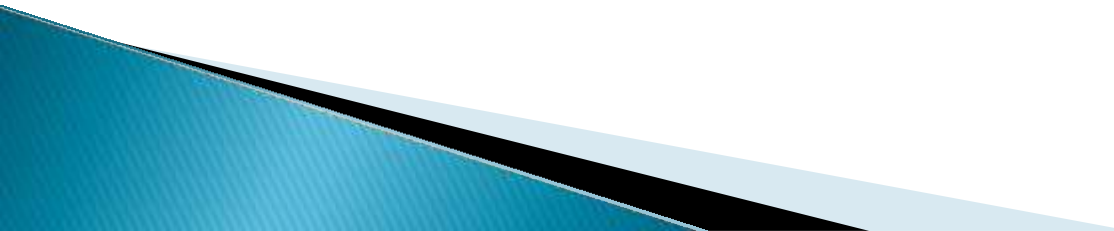
▮ 4. Recursively, merge the two halves in a sorted manner, so that only one sorted array is left:

▮ merge(array, l, m, r)

# Example

```
#include<stdio.h>
void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);
void main()
{
int a[30],n,i;
printf("Enter no of elements:");
scanf("%d",&n);
printf("Enter array elements:");
for(i=0;i<n;i++)
scanf("%d",&a[i]);
mergesort(a,0,n-1);
printf("\nSorted array is :");
for(i=0;i<n;i++)
printf("%d ",a[i]);
}
```

```
void mergesort(int a[],int i,int j)  
{  
int mid;  
if(i<j)  
{  
mid=(i+j)/2;  
mergesort(a,i,mid); //left recursion  
mergesort(a,mid+1,j); //right recursion  
merge(a,i,mid,j); //merging of two sorted sub- arrays  
}  
}
```



```

void merge(int a[],int first,int mid,int last)
{
    int b[50]; //array used for merging int i,j,k;
    int i,j,k;
    i=first;
    j=mid+1;
    k=first;
    While(i<=mid&& j<=last)
    {
        if(a[i]<=a[j])
            b[k++]=a[i++];
        else
            b[k++]=a[j++];
    }
    If(i>mid)
    {
        while(j<=last)
            b[k++]=a[j++];
    }
    Else
    {
        while(i<=mid)
            b[k++]=a[i++];
    }
    For(i=first;i<=last;i++)
        a[i]=b[i];
}

```



DOSBox 0.74, Cpu speed: max 100% cycles, F

Enter no of elements:5

Enter array elements:990

3

80

8

800

Sorted array is :3 8 80 800 990

# Quicksort

- Quicksort is a divide-and-conquer algorithm. It works by selecting a **'pivot' element** from the array and **partitioning the other elements into two sub-arrays**, according to whether they are less than or greater than the pivot.
  - There are many different versions of quickSort that pick pivot in different ways.
    - ❖ Always pick first element as pivot.
    - ❖ Always pick last element as pivot (implemented below)
    - ❖ Pick a random element as pivot.
    - ❖ Pick median as pivot.
- 

▮ The key process in quickSort is **partition()**.

▮ Target of partitions is, given an array and an element  $x$  of array as pivot, put  $x$  at its correct position in sorted array and put all smaller elements (smaller than  $x$ ) before  $x$ , and put all greater elements (greater than  $x$ ) after  $x$ .

# Example

Consider an array having 6 elements

5      2      6      1      3      4

Arrange the elements in ascending order using  
quick sort algorithm

This is our unsorted array

|               |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|
| Array index   | p | 1 | 2 | 3 | 4 | 5 |
| Array element | 5 | 2 | 6 | 1 | 3 | 4 |



Left

Initially pointing to the  
First element of the array



Right

Initially pointing to the  
Last element of the array

Initially pointing to the  
First element

Pivot



This is our unsorted array

Array index

0

1

2

3

4

5

Array element

5

2

6

1

3

4

We will quick sort this array



Left



Right

Initially pointing to the  
First element of the array

Initially pointing to the  
Last element of the array

Remember this rule:

All element to the **RIGHT** of pivot be **GREATER** than pivot.

All element to the **LEFT** of pivot be **SMALLER** than pivot.

Pivot



|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| 0        | 1        | 2        | 3        | 4        | 5        |
| <b>5</b> | <b>2</b> | <b>6</b> | <b>1</b> | <b>3</b> | <b>4</b> |

Left



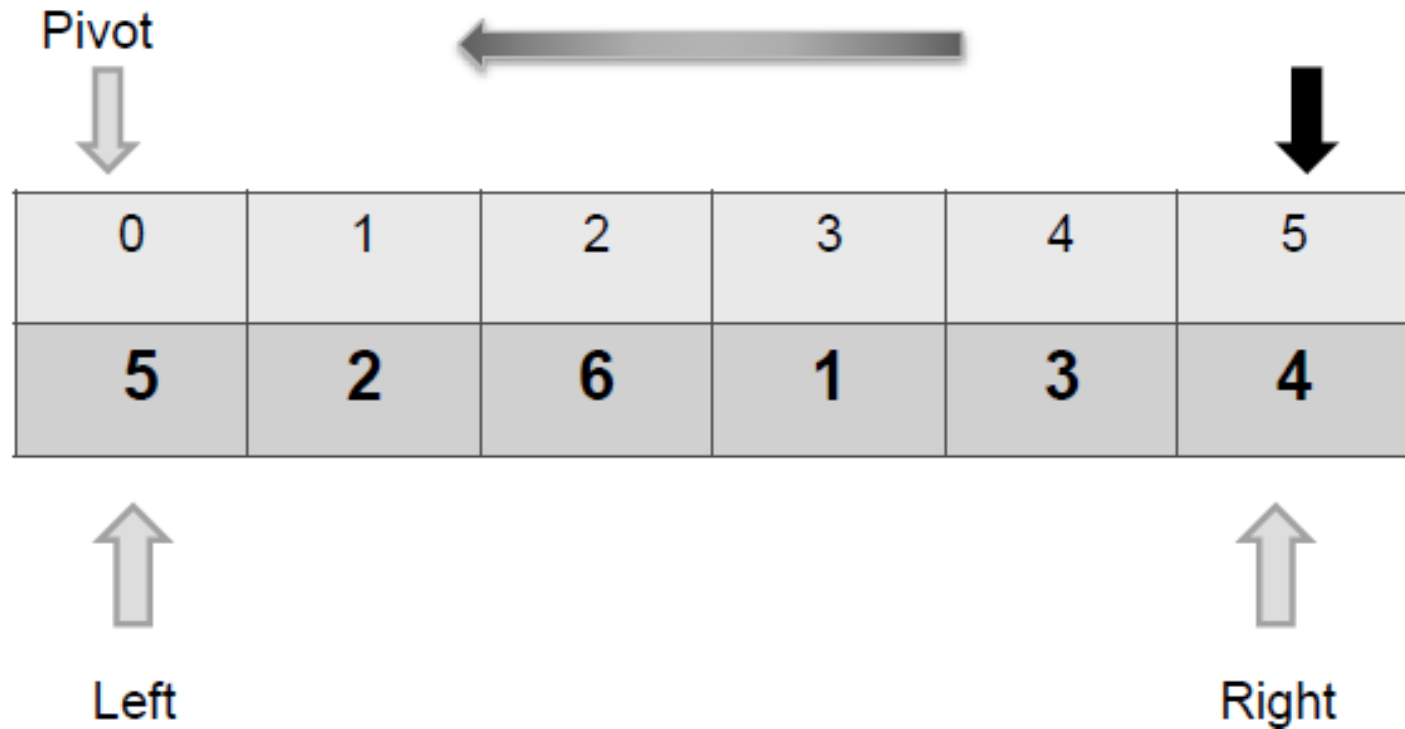
Right



As the pivot  
is pointing at  
left

And move towards left

So we will start  
from right

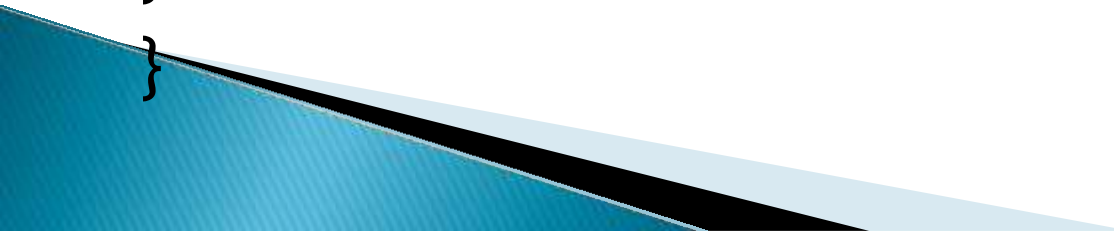


# Example :

```
#include<stdio.h>
void quicksort(int number[25],int first,int last)
{
int i, j, pivot, temp;
if(first<last)
{
pivot=first;
i=first;
j=last;
while(i<j)
{
while(number[i]<=number[pivot]&& i<last)
i++;
while(number[j]>number[pivot])
j--;
```



```
if(i < j)
{
temp=number[i];
number[i]=number[j];
number[j]=temp;
}
}
temp=number[pivot];
    number[pivot]=number[j];
number[j]=temp;
    quicksort(number,0,j-1);
    quicksort(number,j+1,last);
}
}
```

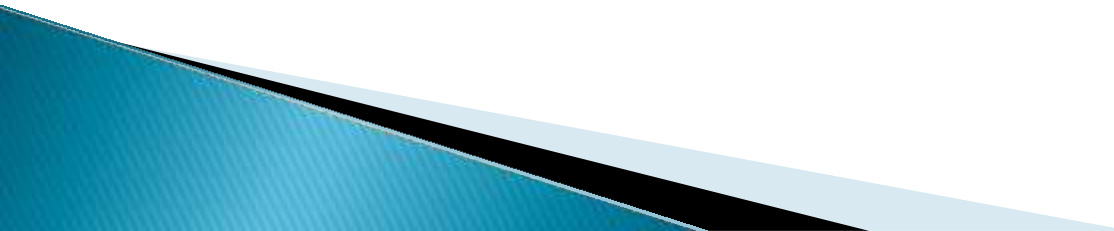
A decorative graphic in the bottom-left corner consisting of overlapping blue and black geometric shapes, possibly representing a stylized landscape or abstract design.

```
void main()
{
int i,n, number[25];
printf("How many elements are u going to enter?:
");
scanf("%d",&n);
printf("Enter elements: ");
for(i=0;i<n;i++)
scanf("%d",&number[i]);
quicksort(number,0,n-1);
printf("Order of Sorted elements: ");
for(i=0;i<n;i++)
printf(" %d",number[i]);
}
```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program:

```
How many elements are u going to enter?: 5
Enter elements: 78
8
23
4
7
Order of Sorted elements: 4 7 8 23 78_
```

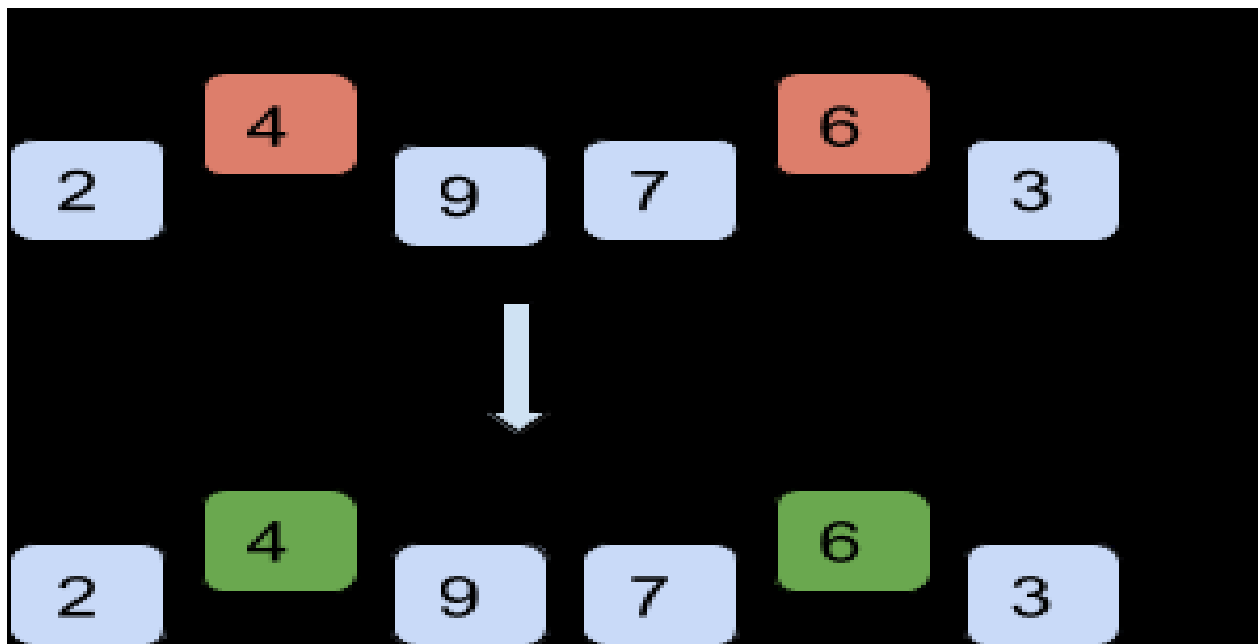
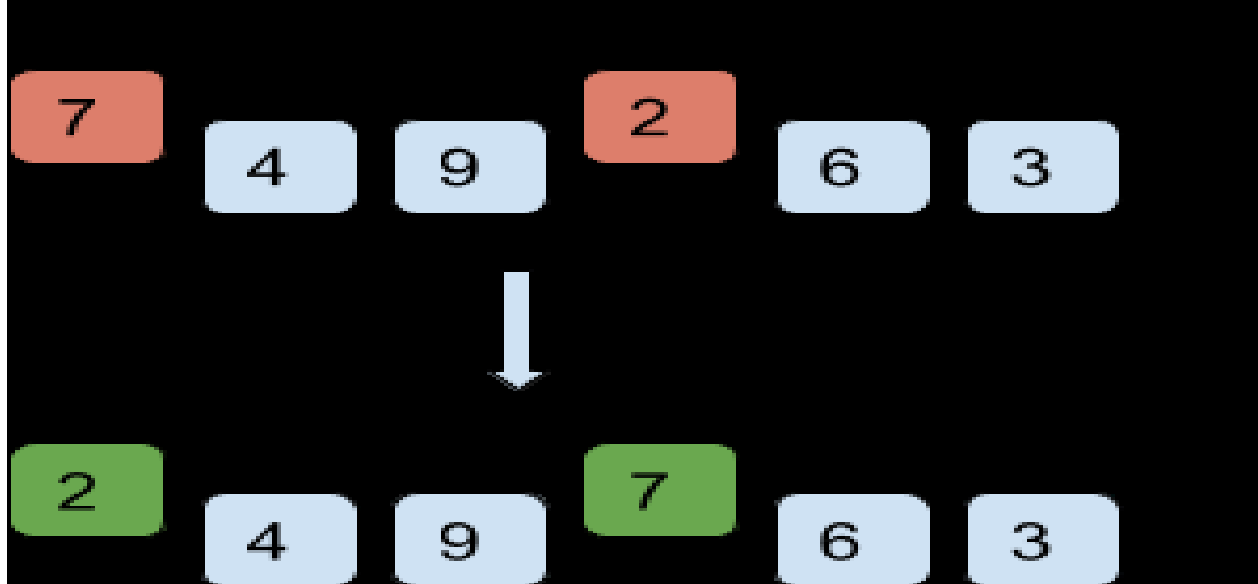
# Shell Sort

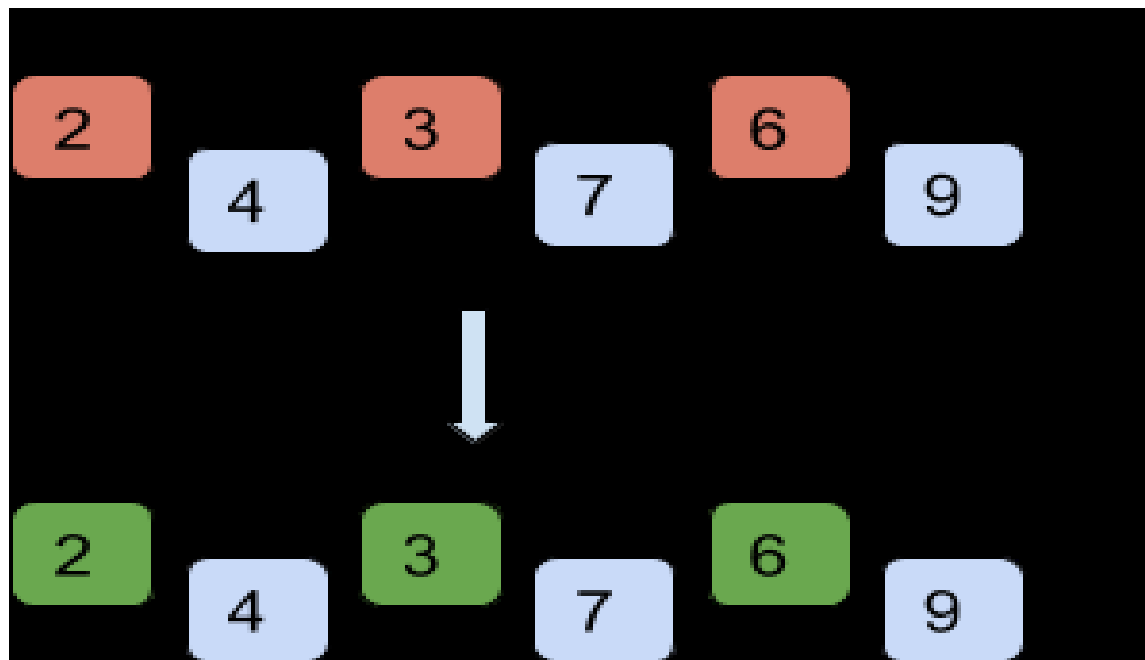
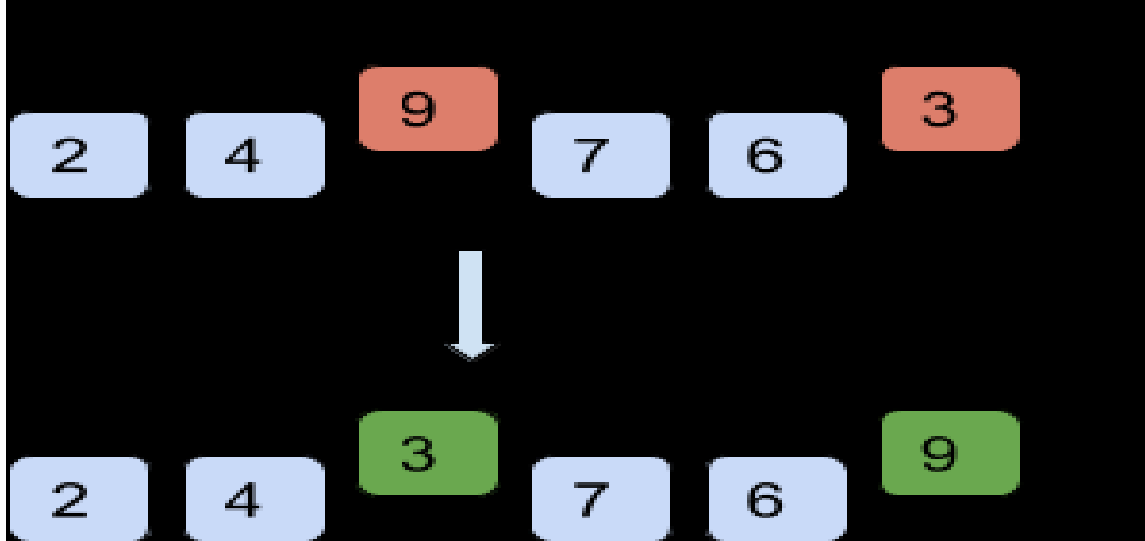
- ▮ Shellsort, also known as **Shell sort** or **Shell's method**, is an **in- place comparison sort**.
  - ▮ It can either be seen as a generalization of sorting by exchange (bubble sort) or sorting by insertion (insertion sort).
  - ▮ The method starts by sorting elements far apart from each other and progressively reducing the gap between them.
  - ▮ Starting with far apart elements can move some out-of-place elements into position faster than a simple nearest neighbor exchange.
- 

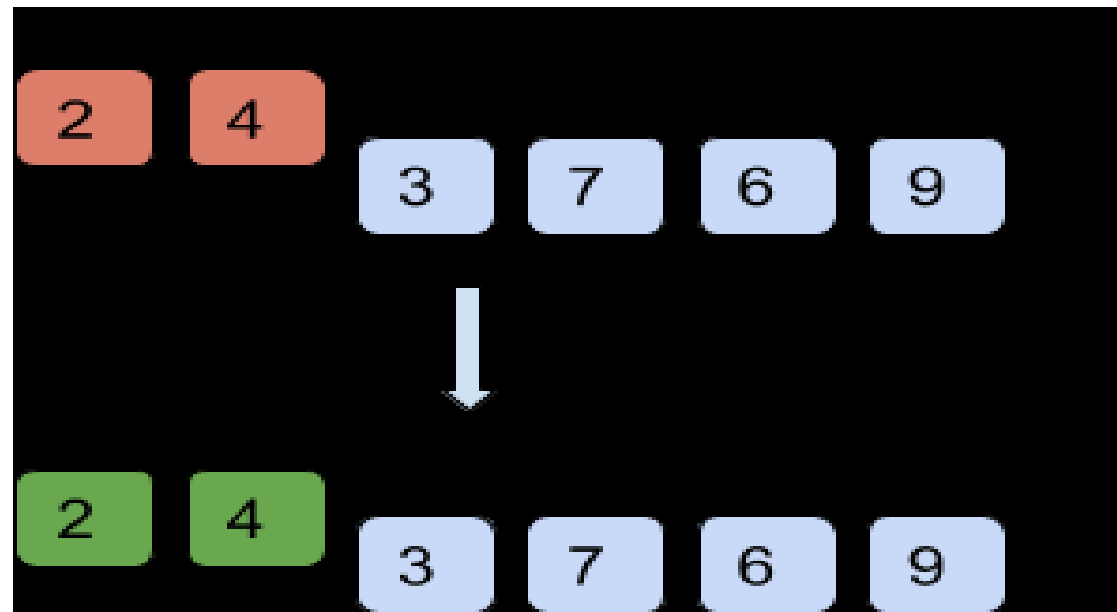
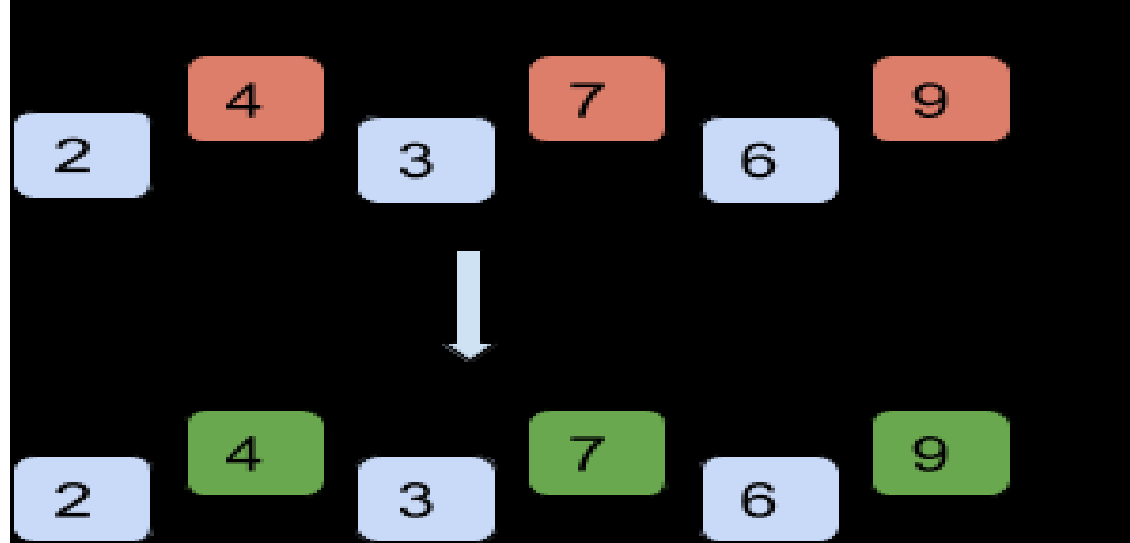
## Algorithm of Shell Sort

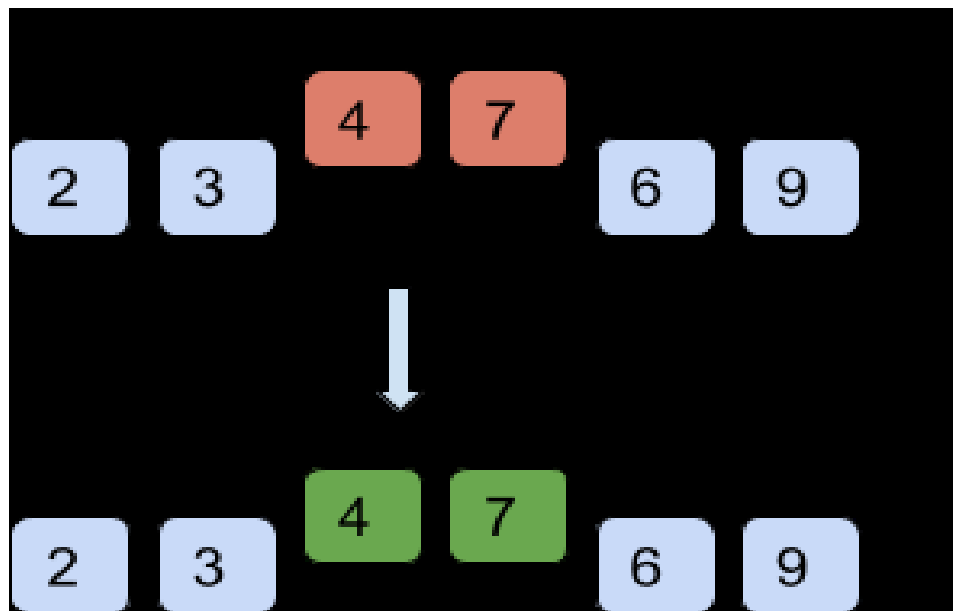
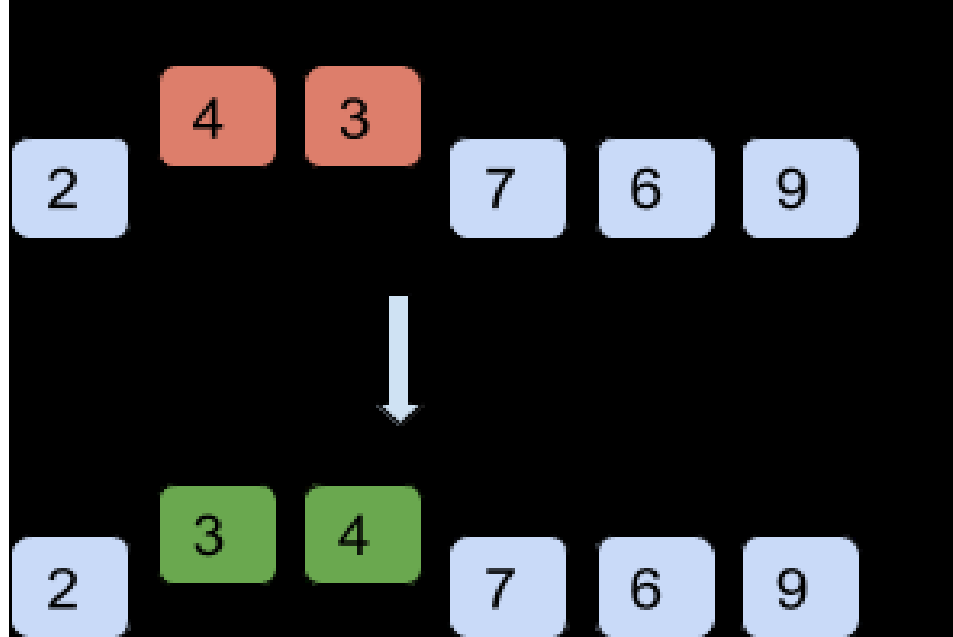
1. Initialize the gap size.
2. Divide the array into subarrays of equal gap size.
3. Apply insertion sort on the subarrays.
4. Repeat the above steps until the gap size becomes 0 resulting into a sorted array.

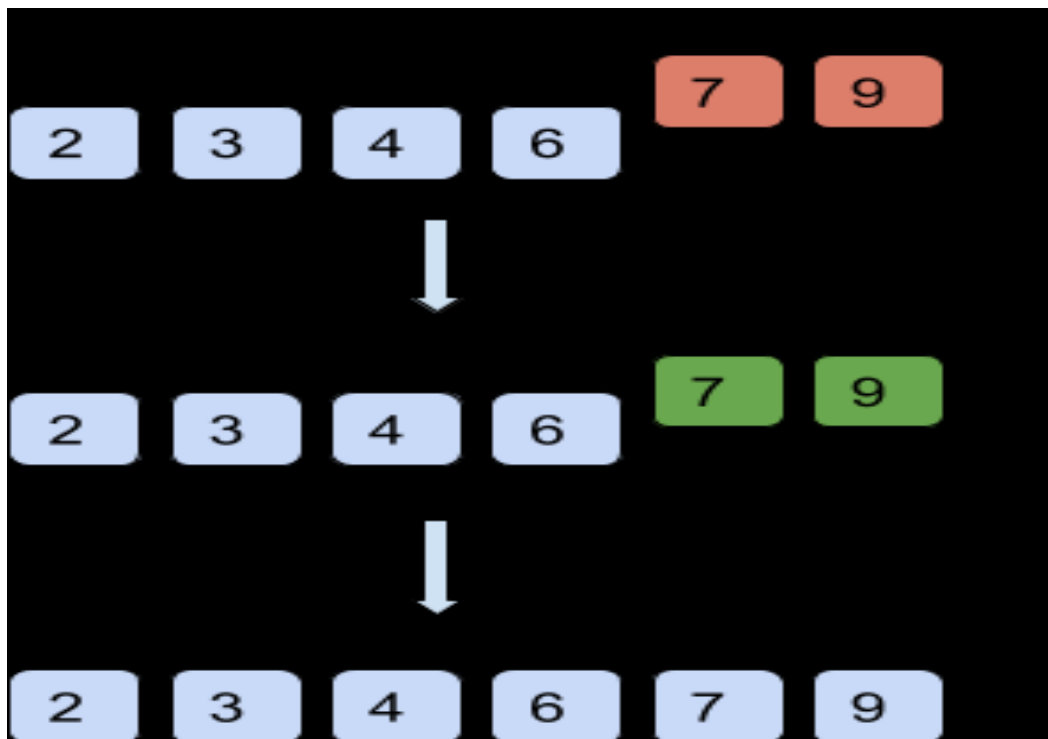
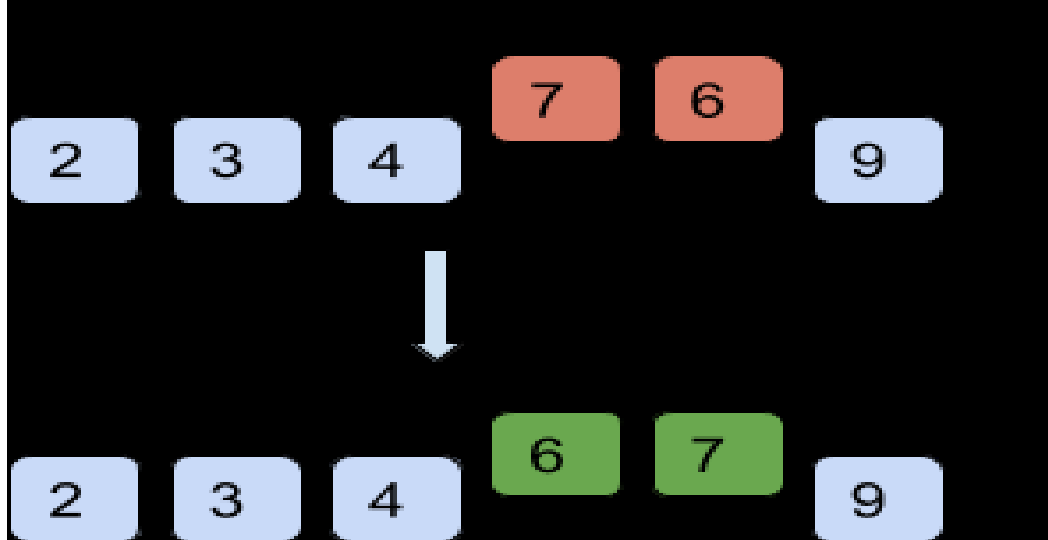
Find below the working of shell sort for the example list = [7, 4, 9, 2, 6, 3] with starting gap size = 3 and reducing the gap size by 1 after every iteration till the gap size is greater than 0.







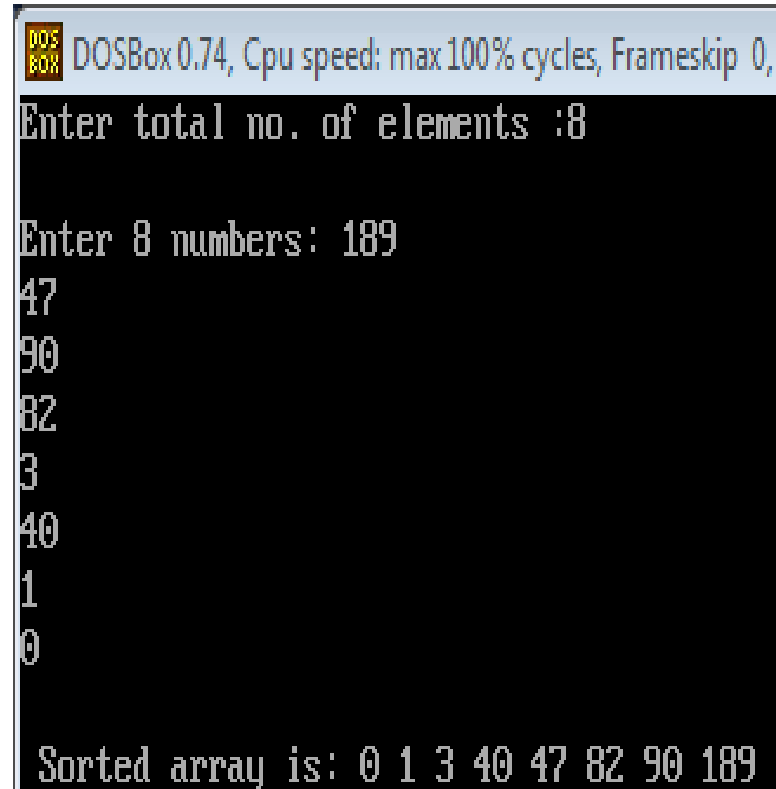




# Example:

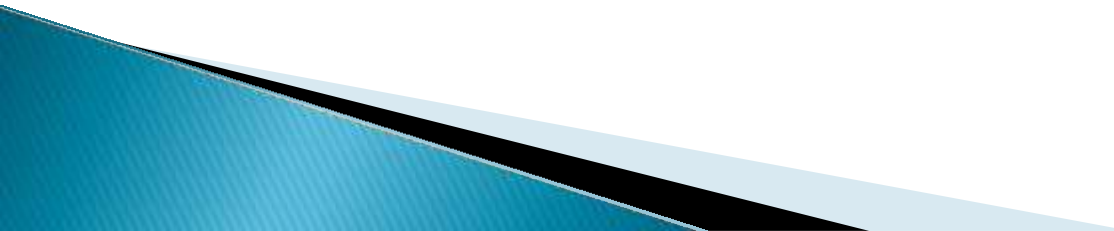
```
#include <stdio.h>
void shellsort(int arr[], int num)
{
    int i, j, k, tmp;
    for (i = num / 2; i > 0; i = i / 2)
    {
        for (j = i; j < num; j++)
        {
            for (k = j - i; k >= 0; k = k - i)
            {
                if (arr[k+i] >= arr[k])
                    break;
                else
                {
                    tmp = arr[k];
                    arr[k] = arr[k+i];
                    arr[k+i] = tmp;
                }
            }
        }
    }
}
```

```
void main()
{
    int arr[30]; int k, num;
    printf("Enter total no. of elements : ");
    scanf("%d", &num);
    printf("\nEnter %d numbers: ", num);
    for (k = 0 ; k < num; k++)
    {
        scanf("%d", &arr[k]);
    }
    shellsort(arr, num);
    printf("\n Sorted array is: ");
    for (k = 0; k < num; k++)
        printf("%d ", arr[k]);
}
```



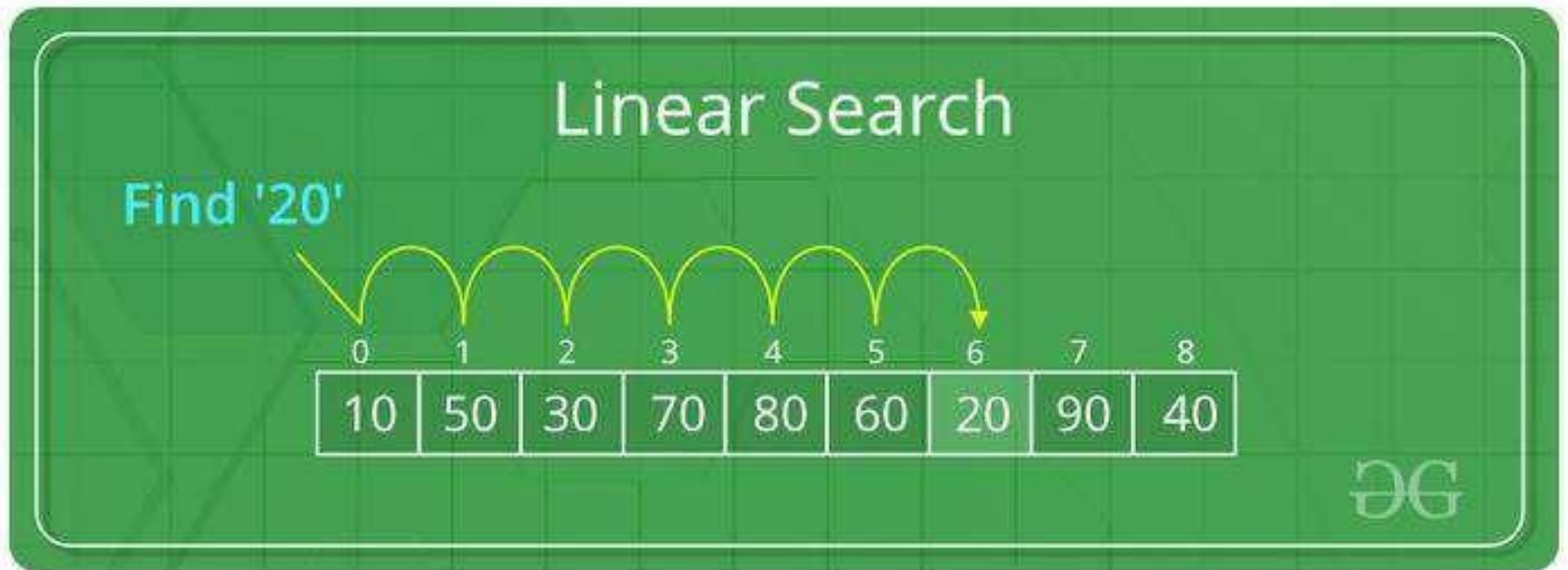
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0,  
Enter total no. of elements :8  
Enter 8 numbers: 189  
47  
90  
82  
3  
40  
1  
0  
Sorted array is: 0 1 3 40 47 82 90 189

# Searching Techniques

- ▮ Searching is the process of finding some particular element in the list.
  - ▮ The process of identifying or finding a particular record is called Searching.
  - ▮ If the element is present in the list, then the process is called successful and the process returns the location of that element, otherwise the search is called unsuccessful.
  - ▮ There are two popular search methods that are widely used in order to search some item into the list. However, choice of the algorithm depends upon the arrangement of the list.
- 

**1. Sequential Search:** In this, the list or array is traversed sequentially and every element is checked. **For example: Linear Search.**

Linear Search to find the element "20" in a given list of numbers



**2. Interval Search:** These algorithms are specifically designed for searching in sorted data-structures. These type of searching algorithms are much more efficient than Linear Search as they repeatedly target the center of the search structure and divide the search space in half. **For Example: Binary Search.**

**Binary Search to find the element "23" in a given list of numbers**



## 1. Sequential Searching

- Sequential search is also called as Linear Search.
- Sequential search starts at the beginning of the list and checks every element of the list.
- It is a basic and simple search algorithm.
- Sequential search compares the element with all the other elements given in the list. If the element is matched, it returns the value index, else it returns -1.

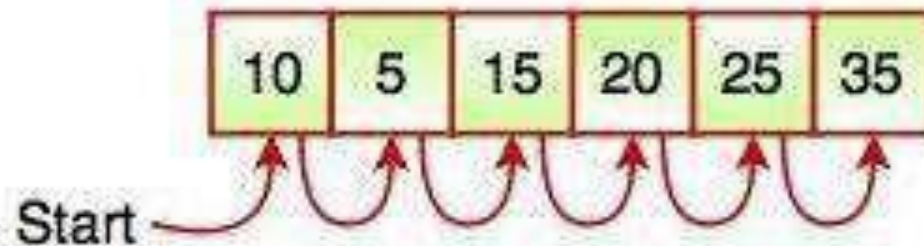


Fig. Sequential Search

▮ The above figure shows how sequential search works. It searches an element or value from an array till the desired element or value is not found. If we search the element 25, it will go step by step in a sequence order.

▮ It searches in a sequence order. Sequential search is applied on the unsorted or unordered list when there are fewer elements in a list.

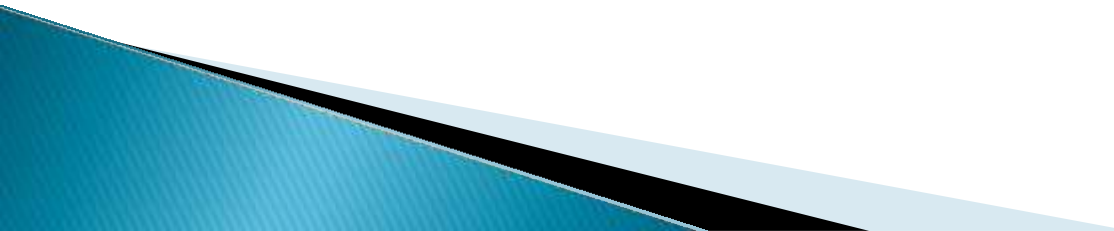
### Linear Search

#### □ Algorithm:

- **Step1:** Start from the leftmost element of array and one by one compare x with each element of array.
- **Step2:** If x matches with an element, return the index.
- **Step3:** If x doesn't match with any of elements, return -1.

# Example :

```
#include <stdio.h>
void main()
{
    int arr[50], search, cnt, num;
    printf("Enter the number of elements in
        array\n");
    scanf("%d",&num);
    printf("Enter %d integer(s)\n", num);
    for (cnt = 1; cnt <= num; cnt++)
        scanf("%d", &arr[cnt]);
    printf("Enter the number to search:\n");
```



```
scanf("%d", &search);
for (cnt = 0; cnt < num; cnt++)
{
    if (arr[cnt] == search) /* if required
        element found */
    {
        printf("%d is present at index of %d.\n",
            search, cnt+1);
        break; }
    }
    if (cnt >= num)
        printf("%d is not present in array.\n",
            search);
}
```

DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Prog

Enter the number of elements in array

5

Enter 5 integer(s)

48

24

78

90

33

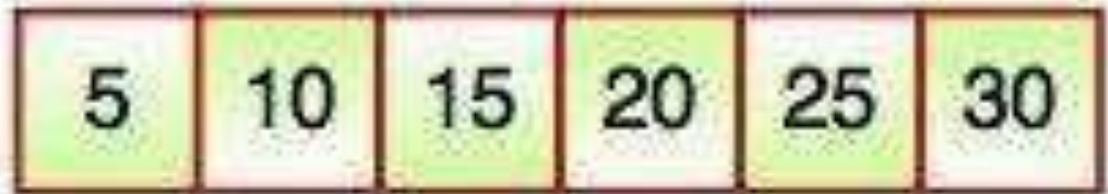
Enter the number to search:

90

90 is present at index of 4.

## 2. Binary Searching

- Binary Search is used for searching an element in a sorted array.
- It is a fast search algorithm with run-time complexity of  $O(\log n)$ .
- Binary search works on the principle of divide and conquer.
- This searching technique looks for a particular element by comparing the middle most element of the collection.
- It is useful when there are large number of elements in an array.



|   |    |    |    |    |    |
|---|----|----|----|----|----|
| 5 | 10 | 15 | 20 | 25 | 30 |
|---|----|----|----|----|----|

- The above array is sorted in ascending order. As we know binary search is applied on sorted lists only for fast searching.

For example, if searching an element 25 in the 7-element array, following figure shows how binary search works:



Fig. Working Structure of Binary Search

Binary searching starts with middle element.

If the element is equal to the element that we are searching then return true. If the element is less than then move to the right of the list or if the element is greater than then move to the left of the list. Repeat this, till you find an element.

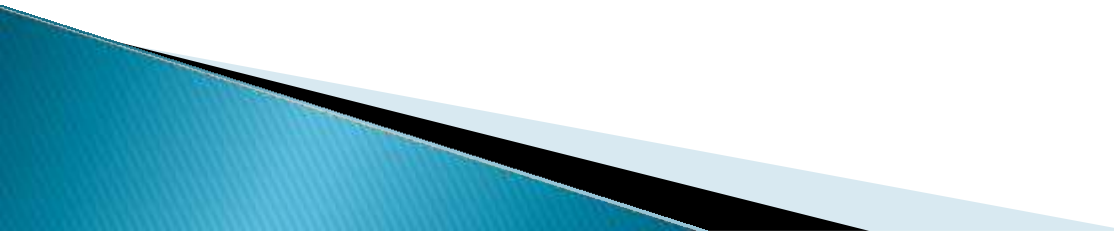
## Binary Search

### □ Algorithm:

- Step1: Compare  $x$  with the middle element.
- Step2: If  $x$  matches with middle element, we return the mid index.
- Step3: Else If  $x$  is greater than the mid element, search on right half.
- Step4: Else If  $x$  is smaller than the mid element. search on left half.

# Example :

```
#include<stdio.h>
#include<conio.h>
void main()
{
int f, l, m, size, i, sElement, list[50]; //int f, l ,m :
    First, Last, Middle
printf("Enter the size of the list: ");
    scanf("%d",&size);
printf("Enter %d integer values : \n", size);
for (i = 0; i < size; i++)
    scanf("%d",&list[i]);
printf("Enter value to be search: ");
    scanf("%d", &sElement);
f = 0;
l = size - 1;
```



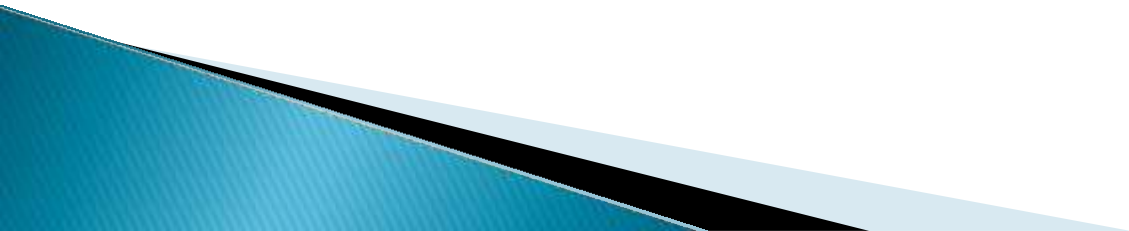
```
m = (f+l)/2;
while (f <= l)
{
if (list[m] < sElement)
f = m + 1;
else if (list[m] == sElement)
{
printf("Element found at index %d.\n",m);
break;
}
else
l = m - 1;
m = (f + l)/2;
}
if (f > l)
printf("Element Not found in the list.");
}
```

DOSBox 0.74, Cpu speed: max 100% cycles, Fr

```
Enter the size of the list: 5
Enter 5 integer values :
10
20
45
78
90
Enter value to be search: 20
Element found at index 1.
```

# Introduction to Hash Tables

A **hash table** (or hash map) is a data structure that implements an associative array, a structure that can map keys to values. It provides an efficient way to store and retrieve data based on a key.



## ❑ Advantages of Hash Tables

**Fast Access:** Average-case time complexity for search, insert, and delete operations is  $O(1)$ , making hash tables very efficient for lookups.

**Flexible Key Types:** Keys can be of various types (strings, numbers, etc.), provided a suitable hash function is defined.

## ❑ Disadvantages of Hash Tables

**Space Inefficiency:** If the table is sparsely populated, it can waste memory.

**Collision Resolution Overhead:** Handling collisions can add complexity and overhead, especially if there are many collisions.

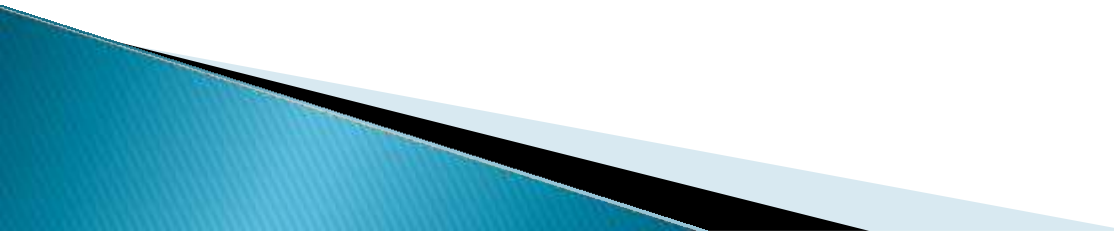
**Poor Performance with High Load Factor:** As the number of entries increases relative to the number of buckets, performance can degrade, leading to longer search times.



## What is a Symbol Table?

A **symbol table** is a data structure used to store information about identifiers (like variables, functions, objects) used in a program.

It maps **symbols (keys)** to associated **information (values)**, such as data type, scope, memory location, etc.



## ◆ Common Use Cases

**Compilers:** To track variables, functions, classes, etc.

**Interpreters:** To manage runtime environments

**Assemblers:** To hold labels and variables

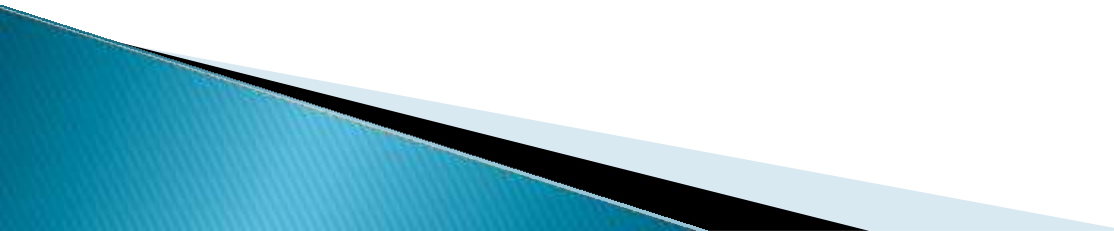
**Databases:** As a mapping between field names and data

## ◆ Purpose of a Symbol Table

To support **efficient lookup** of identifiers

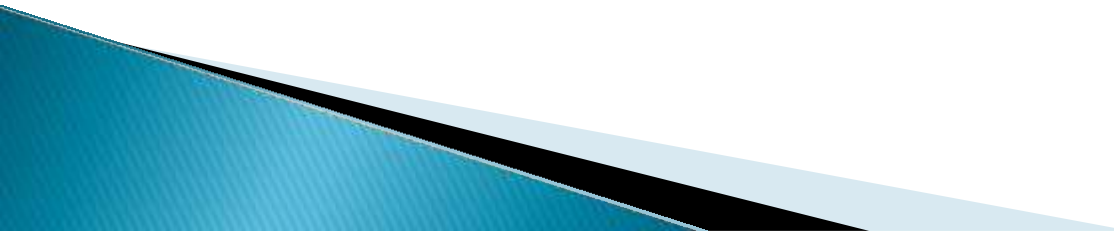
To ensure **semantic correctness** during compilation or interpretation

To help in **code generation and optimization**



# Hashing Function

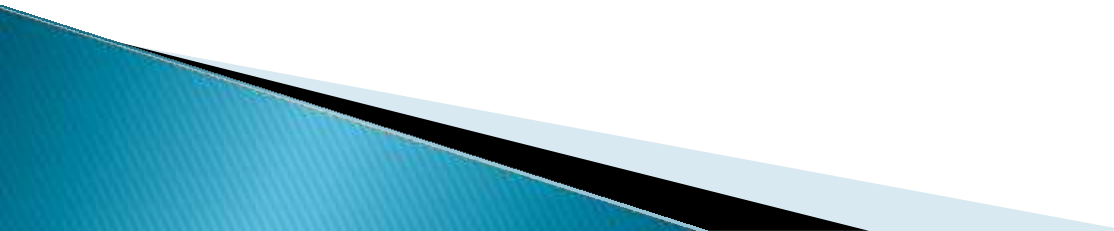
A **hashing function** is a mathematical algorithm that transforms input data (often called a "key") into a fixed-size value, typically referred to as a hash code or hash value. This hash value serves as an index that allows for efficient data retrieval in data structures like hash tables.



# 1. Multiplication Method

▮ The Multiplication Method uses a constant fraction of the key to generate a hash index. It aims to distribute keys more uniformly across the hash table.

## ▮ Steps:

- **Choose a constant  $A$  (between 0 and 1). A common choice is  $A \approx 0.6180339887$  (the inverse of the golden ratio).**
  - **Multiply the key by  $A$ .**
  - **Take the fractional part of the result.**
  - **Multiply by the size of the hash table  $m$ .**
  - **Take the floor of the result to get the index.**
- 

Formula:  $\text{index} = \lfloor m \times (k \times A \bmod 1) \rfloor$

Example:

Key: 57

Hash Table Size: 10

Constant A: 0.6180339887

Calculation:

- $57 \times 0.6180339887 = 35.19892457$
- Take the fractional part: 0.1989240
- Multiply by table size:  $10 \times 0.198924 = 1.9892410$
- Take the floor:  $\lfloor 1.98924 \rfloor =$

Result: The index is 1.

## 2. Mid-Square Method

▮ The Mid-Square Method involves squaring the key and then using the middle digits of the squared value to find the hash index.

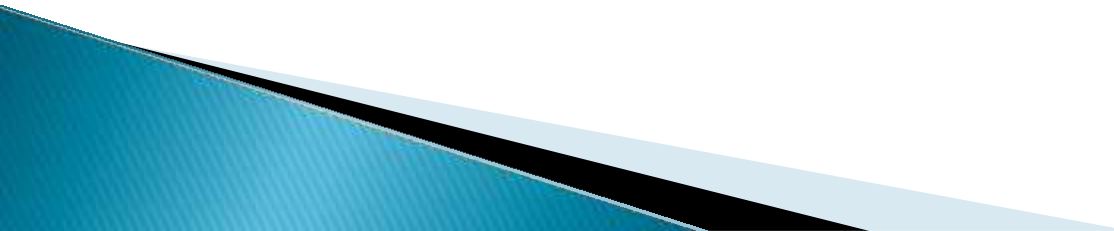
▮ **Steps:**

▮ **Square the key.**

▮ **Convert the square to a string (if necessary).**

▮ **Extract the middle digits from the squared value.**

▮ **Convert those digits to an index by taking the modulus with the size of the hash table.**



## Example:

Key: 57

Hash Table Size: 10

Calculation:

Square the key:  $57 * 57 = 3249$

The middle two digits (in this case) are  
**24.**

Take modulus with table size:  $24 \bmod 10 = 4$

**Result:** The index is 4.

# 3. Division Method

▮ The Division Method uses the modulus operator to find an index for a given key.

▮ **How it Works:**

▮ Take the key and divide it by the size of the hash table.

▮ Use the remainder as the index.

▮ **Formula:**  $\text{index} = \text{key} \bmod m$

▮ **Where:**

▮ key is the value you want to hash.

▮ m is the size of the hash table (should be a prime number for better distribution).

**Example:**

**Key: 57**

**Hash Table Size: 10**

**Calculation:  $57 \bmod$**

**$10 = 7$**

**Result: The index is 7.**

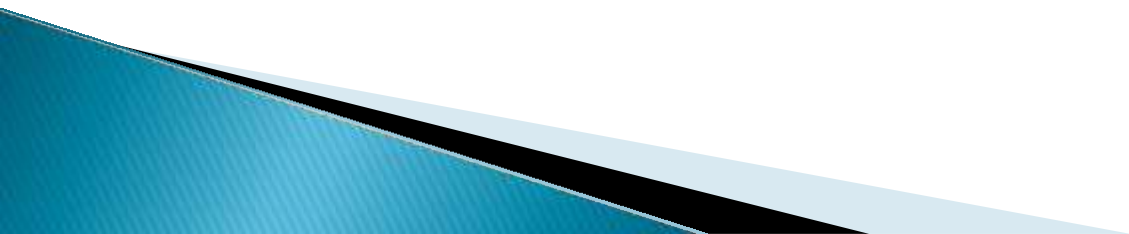
# Applications of Hashing

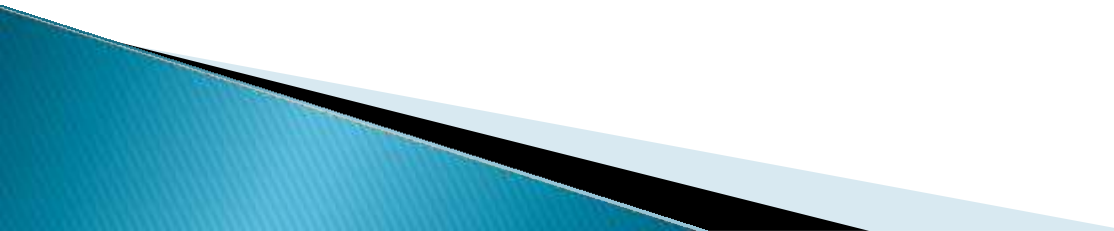
**Hash Tables:** Fast data storage and retrieval using key–value pairs.

**Password Storage:** Securely storing passwords by saving their hash values instead of plaintext.

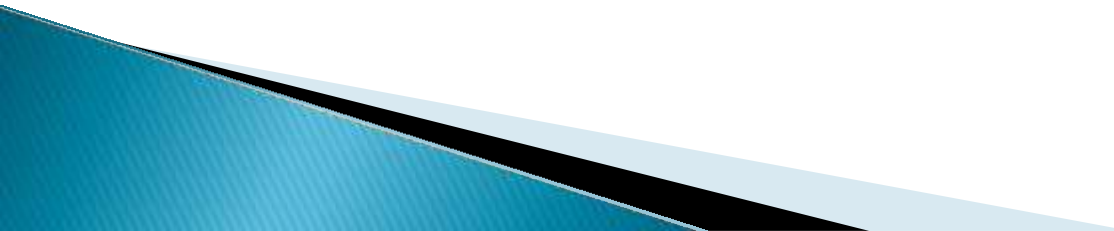
**Digital Signatures:** Ensuring data integrity and authenticity by hashing messages.

**Data Integrity Verification:** Using checksums to verify that data hasn't been altered during transfer.



- Caching:** Quickly retrieving cached data by hashing URLs or data identifiers.
  - Load Balancing:** Distributing requests evenly across servers based on hashed values.
  - Database Indexing:** Creating fast lookup indexes for efficient data retrieval.
  - Duplicate Detection:** Identifying duplicate records by comparing hash values.
  - Object Comparison:** Quickly comparing complex objects using their hash values.
- 

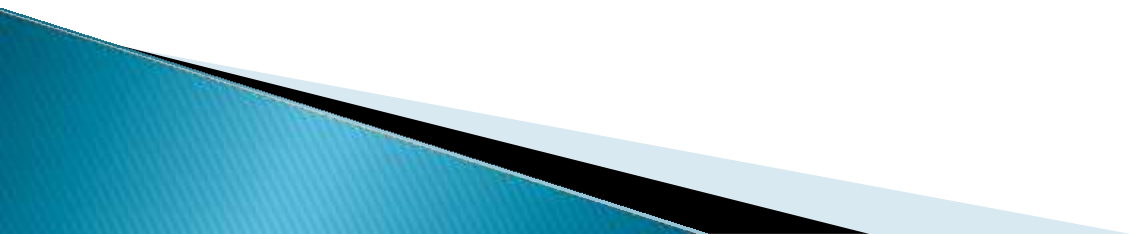
## Collision in hashing:

1. In this, the hash function is used to compute the index of the array.
  2. The hash value is used to store the key in the hash table, as an index.
  3. The hash function can return the same hash value for two or more keys.
  4. When two or more keys are given the same hash value, it is called a **collision**. To handle this collision, we use collision resolution techniques
- 

# Types of Collision Resolution Techniques

Collision resolution techniques There are two types of collision resolution techniques. :

1. Separate chaining (open hashing)
2. Open addressing (closed hashing)



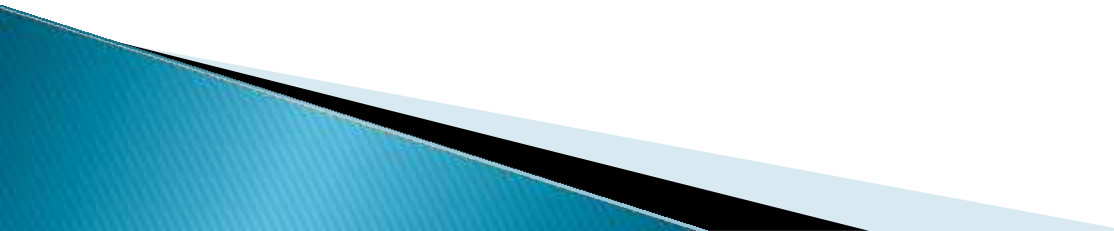
## 1. Separate chaining:

- In this technique, a linked list is created from the slot in which collision has occurred, after which the new key is inserted into the linked list.
- This linked list of slots looks like a chain, so it is called separate chaining.
- It is used more when we do not know how many keys to insert or delete.
- Time complexity:
  1. Its worst-case complexity for searching is  $O(n)$ .
  2. Its worst-case complexity for deletion is  $O(n)$ .

## ❑ Advantages of separate chaining :

1. It is easy to implement.
2. The hash table never fills full, so we can add more elements to the chain.
3. It is less sensitive to the function of the hashing.

## ❑ Disadvantages of separate chaining :

1. In this, cache performance of chaining is not good.
  2. The memory wastage is too much in this method.
  3. It requires more space for element links.
- 

## 2. Open addressing:

- Open addressing is collision-resolution method that is used to control the collision in the hashing table.
- There is no key stored outside of the hash table.

Therefore, the size of the hash table is always greater than or equal to the number of keys.

- It is also called closed hashing.
- The following techniques are used in open addressing:

1. Linear probing
2. Quadratic probing
3. Double hashing

- **Linear probing** – Here, the next probe interval is fixed to 1. It supports the best caching but miserably fails at clustering.
  - **Quadratic probing** – the probe distance is calculated based on the quadratic equation. This is considerably a better option as it balances clustering and caching.
  - **Double hashing** – Here, the probing interval is fixed for each record by a second hashing function. This technique has poor cache performance although it does not have any clustering issues.
- 